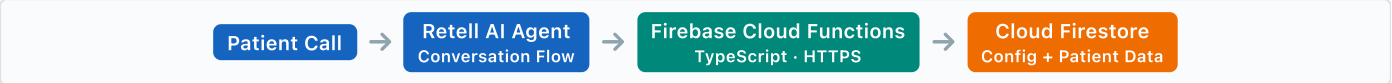
Summit Dermatology — Voice AI Agent

Architecture Document · Confido Health · FDE Assessment · Feb 2026

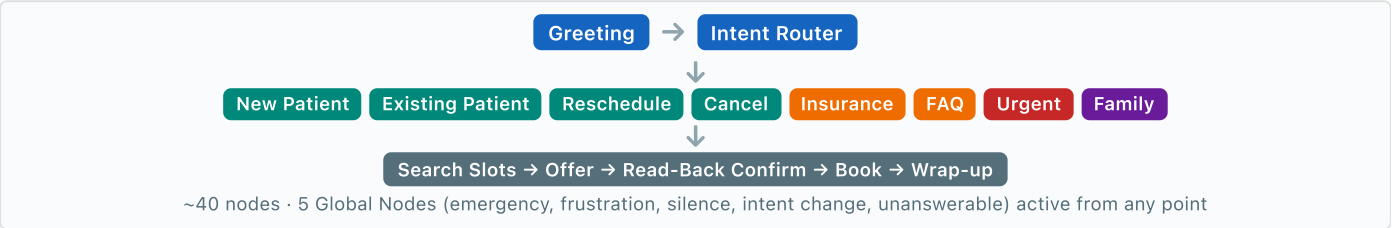
A. Agent Structure

Prompt & Conversation Node Structure

I chose **Retell AI's Conversation Flow** over a single-prompt approach after researching when each is most effective. Since this agent covers multiple communication paths and edge cases, the node-based flow keeps it focused on one task at a time, preventing context loss across long conversations. Each node has its own scoped prompt, making behavior predictable and easy to debug.



Conversation Flow Overview



State Management

The agent uses Retell's **dynamic variables** to store data collected during the call (`first_name`, `dob`, `patient_id`, `selected_slot_id`). Each node builds on what was previously collected without re-asking the caller. Persistent data (locations, providers, patients) is stored in **Cloud Firestore** for fast querying during calls.

Escalation Logic

Escalation to a human is handled through specific nodes. While simulated in this demo, certain paths lead to human handoff:

B. Guardrails

Preventing Hallucinations

Minimized through **global context and constraints** in the agent's prompt. The Conversation Flow itself is a guardrail: each node focuses on a single action, so the agent cannot drift into generating wrong information. Appointment times only come from `getAvailableSlots` — never invented.

Ensuring Data Clarity

Critical fields (DOB, phone) are **repeated back in natural language** for caller confirmation. Backend Cloud Functions include **multi-format parsers** that normalize any input (e.g., "July 22, 1990" → `07-22-1990`). In production, an AI layer could further format ambiguous inputs.

Handling Failure Cases

Two strategies: **(1)** Re-ask the caller for corrected info and retry the lookup. **(2)** A Global Node detects repeated failures and suggests transferring to human staff, preventing unproductive loops.

C. What Will Break in Production

Issue	Risk	Problem	Mitigation
Pronunciation & spelling	HIGH	AI doesn't always understand callers — mic quality and accents compound this. Phonetic spelling often transcribes incorrectly.	Spell one letter at a time. In production, consider DTMF (keypad) for critical fields.
DOB format variance	HIGH	Every person says their DOB differently: "March third", "3/3/80", "03-03-1980". System must handle all formats.	Multi-format normalizer in Cloud Functions parses any format to `MM-DD-YYYY`.
Phone number dictation	MED	Callers dictate phone numbers differently — grouping, speed, and pronunciation vary widely.	Read-back confirmation; backend validates 10-digit format before accepting.
Simulated slot data	MED	Current system uses mocked availability. Production needs real-time EHR integration to avoid stale slots.	Replace Firestore seed with live EHR feed. Cloud Function interface stays the same.
Frustrated callers	MED	Callers who become frustrated need graceful handling to avoid negative experiences.	Global Node detects frustration, responds with empathy, offers immediate transfer.
Re-ask loops	LOW	Without enough exit conditions, the agent could get stuck repeatedly asking the same question.	Max 2 clarification attempts per node; after that, offer transfer to staff.

D. Scaling from 5 to 40 Locations

Avoiding Copy-Paste

Each location gets its own **phone number** (or an identification step) mapping to a `location_id` dynamic variable. The same agent serves all locations — decision nodes and Cloud Functions route to the appropriate data based on that ID. No agent duplication needed.

Separating Config from Core Logic

Core Logic (shared, 1 copy)

- Retell Conversation Flow (all nodes, prompts, transitions)
- Firebase Cloud Functions (5 HTTPS endpoints)
- Global prompt & 5 Global Nodes

+

Configuration (per location)

- Firestore `locations/{id}` — hours, address, providers, insurance
- Firestore `availableSlots` — per-location availability
- Retell phone number → `location_id` mapping

Core logic is identical across all locations. The backend manages what each offers — providers, hours, parking, insurance. Location-specific workflows can be added via decision nodes without affecting others.

Preventing Drift & Version Control

All locations share the **exact same agent and Cloud Functions**. Differences live in the database only. Updates are deployed once and propagate everywhere. Adding a new location requires only a Firestore document and a phone number — **no code changes**:

Step	Action	Time
1	Create Firestore doc at `locations/{new-id}` with hours, address, providers, insurance	~10 min
2	Buy/assign phone number in Retell, set `location_id = new-id`	~5 min
3	Seed availability data in `availableSlots` (or connect to real EHR)	~5 min